

Une sous-etape supprimee, et la provenance reprise

Force Tracker — 12/09/2026, ft -v1197. Neuvieme document de la serie. Michel : « *continue selon le decoupage, une sous-etape a la fois* ». Deux choses se sont passees : la sous-etape suivante s'est revelee **vide**, et la suivante encore a ete livree.

LE FAIT PRINCIPAL DE CETTE SESSION

La sous-etape que j'allais prendre n'avait rien a extraire — mesure avant la moindre ligne de code. Et l'erreur qui l'avait fait entrer dans le plan vaut plus que la sous-etape elle-meme : **j'avais verse au decoupage tout ce qu'un compteur defaillant avait rate, sans jamais demander, site par site, s'il etait DUPLIQUE.**

Reparer l'instrument et refaire l'inventaire sont deux gestes differents. J'avais fait le premier en croyant avoir fait le second.

1. Le test d'entree qui manquait

Le decoupage de 1b et 3 comptait **10 sous-etapes**. Chacune etait decrite par son perimetre, ses sites, son instantane, ses mutations et son rollback — mais **aucune ne disait combien de COPIES elle supprimait**. C'est pourtant la seule question qui decide si une extraction a un sens.

UNE EXTRACTION EXIGE AU MOINS DEUX COPIES

Sortir un proprietaire qui n'a qu'un **seul appelant** n'est pas du rangement : c'est de la complexite ajoutee sans contrepartie, et une regle du depot l'interdit explicitement (*la gouvernance sert le produit, jamais l'inverse*).

Le test est desormais ecrit dans le document : **compter les copies AVANT de decrire la sous-etape.**

2. 1b-iv (l'export CSV) : ecartee, et elle reste ecrite

```
// 1b-iv — CE QUE LE PLAN ANNONCAIT COMME « la moins risquee du lot »

const NUTRI_COLONNES = ['date','repas','aliment','quantite','unite', ... ];
async function exportNutritionCsv(){
  const L = (S.foodLog||[]).slice().sort(...).map(e => ({ ...13 champs... }));
  await _csvFichier(NUTRI_COLONNES, L, 'nutrition', 'lignes');
}

// Mesure : cette forme existe UNE SEULE FOIS. Aucun import ne la relit.
// Et _csvFichier(colonnes, lignes, ...) est DEJA le proprietaire commun :
// il sert l'export nutrition ET l'export poids.
```

ce que le plan annonçait	ce que la MESURE dit
« la ligne CSV du journal nutrition, 13 colonnes aux noms français », decrite comme la sous-etape la moins risquee du lot	la forme existe une seule fois dans tout le code servi (NUTRI_COLONNES : 2 occurrences, la declaration et son unique usage). Aucun import ne la relit.
« prerequis absolu : AUCUNE sonde aujourd'hui »	faux aussi : un temoin conduit vraiment exportNutritionCsv(), intercepte la remise du fichier et lit le CSV produit (echappement de la virgule, BOM, provenance, ordre)
—	et le partage etait DEJA fait au bon niveau : _csvFichier(...) est le proprietaire commun, appele par l'export nutrition et l'export poids (3 occurrences)

=> **L'etiquette « instantane » etait fausse DES DEUX COTES — et c'est le miroir exact de la version precedente**, ou elle annonçait « couvert » pour une sonde qui ne couvrait rien. **Dans les deux sens, l'etiquette ne remplace pas l'ouverture du fichier.**

ELLE N'EST PAS EFFACEE DU PLAN, ELLE Y RESTE AVEC SA RAISON

Une sous-etape **supprimee** ne laisse qu'un trou dans une liste — c'est-a-dire exactement a quoi ressemble un oubli. Quelqu'un la remettrait dans six mois.

Elle reste donc ecrite **a sa place dans l'ordre**, marquee ECARTEE, **avec la mesure qui l'a tuee**. *C'est la meme regle qui veut qu'un retrait volontaire s'ecrive comme un ajout.*

Les 8 autres sous-etapes ont ete auditees au meme test, et toutes tiennent : 1b-ii (3-4 sites) · 1b-iii (2) · 1b-v (2, identiques au caractere pres) · 3-ii/iii/iv/v (les 5 sites « grammes seuls »). **Une seule etait vide sur 10.**

3. 1b-ii : la provenance reprise

Trois champs — sourceId, etat, per100 — recopies depuis une ligne déjà enregistrée, a l'identique par deux portes. Une troisième (quickAddFood) n'en porte que la paire : son per100 lui vient du propriétaire de la sous-etape précédente.

```
// AVANT – le meme triplet, ecrit a l'identique par deux portes

// quickFillFood (« Mes aliments »)
_afSetSrc({saisie:'liste', origine:it.origine||'reprise',
  sourceId:it.sourceId|null, etat:it.etat|null, per100:it.per100|null,
  attendu:...});

// _afSuggPrendreLocale (la recherche dans le journal)
_afSetSrc({saisie:'historique', origine:e.origine||'utilisateur',
  sourceId:e.sourceId|null, etat:e.etat|null, per100:e.per100|null,
  attendu:...});

// APRES – un propriétaire pour ce qui est IDENTIQUE ;
//           ce qui DIVERGE reste visible chez l'appelant

function _srcProvenance(src){
  const s = src || {};
  return { sourceId: s.sourceId || null, etat: s.etat || null, per100: s.per100 || null };
}

_afSetSrc(Object.assign({saisie:'liste', origine:it.origine||'reprise'},
  _srcProvenance(it), {attendu:...}));

_afSetSrc(Object.assign({saisie:'historique', origine:e.origine||'utilisateur'},
  _srcProvenance(e), {attendu:...}));
```

4. La coupe est dictée par les DIVERGENCES, pas par la ressemblance

ON EXTRAIT CE QUI EST IDENTIQUE, ON LAISSE VISIBLE CE QUI DIVERGE

Il était tentant de faire entrer origine et saisie dans le propriétaire : ils sont sur la même ligne, ils ressemblent au reste.

Mesure : ils ne disent pas la même chose aux trois portes.

Les unifier changerait **ce que le journal dit de lui-même** — donc ce n'est pas une extraction, c'est une **decision produit**, et elle revient à Michel. *Un écart qu'on lit dans le code ne se perd pas ; un écart absorbe dans un propriétaire, si.*

porte	origine	saisie	ce que ça veut dire
« Mes aliments »	it.origine 'reprise'	'liste'	elle herite de la source si la ligne en avait une
recherche du journal	e.origine 'utilisateur'	'historique'	même mécanique, mais un autre repli et un autre mot
ajout direct	'reprise' (en dur)	'liste'	elle ignore la source EXPRES — voir ci-dessous

=> **Trois formulations, et au moins une est un choix assumé.** Mesure à la sonde : une ligne venue d'un code-barres (origine: 'off') se reenregistre par la porte directe en 'reprise'. **Ce n'est pas un bug** : la source n'est pas conservée sur les favoris, donc en hériter ferait **affirmer une provenance qu'on n'a pas relue**. *La provenance ne ment pas.*

Un témoin de périmètre lit le **corps** du propriétaire et exige que ni origine ni saisie n'y figurent. Un garde de ce document refuse de le produire si c'était le cas.

5. La sonde était morte, et je l'ai vu AVANT de capturer

```
// LA SONDE QUE J'AI ECRITE D'ABORD — ET QUI NE CONDUISAIT RIEN

S.foodLog = [ ...le cas a mesurer... ];
_afSuggPrendreLocale(0); // <- on croit conduire la porte
// resultat mesure : ABSENT, ABSENT, ABSENT... sur les trois cas.

// Cause : elle lit `_afSuggLoc[i]`, PAS `S.foodLog`.
function _afSuggPrendreLocale(i){
  const e = _afSuggLoc[i]; if(!e) return; // <- elle SORTAIT ici
  ...
}

// Corrige : on remplit par la VRAIE fonction de production.
_afSuggLoc = _afSuggLocales(cs.name);
if(!_afSuggLoc.length) throw new Error('liste locale VIDE');
```

POURQUOI C'EST LE MOMENT QUI COMPTE, PAS L'ERREUR

Le critere de reussite est **binaire** : l'instantane doit etre identique octet pour octet avant et apres. Une sonde qui ne conduit rien reste **identique quoi qu'on fasse au code** — elle aurait donc **valide n'importe quelle extraction**, y compris une fausse.

=> **Un BEFORE capture avec une sonde morte est pire qu'aucun BEFORE**, parce qu'il produit une preuve. Le garde qui l'a attrape est trivial et vaut d'etre garde : *la sonde LEVE si la liste qu'elle estensee conduire est vide*.

6. Et j'ai refait une faute que j'avais moi-meme documentee

En eprouvant les gardes de ce document, j'ai mute deux fichiers de l'application **pendant que la passe de tests tournait**. Le serveur du banc relit les fichiers **a chaque requete** : tout bloc qui demarre pendant la fenetre recoit la version cassee, et rend des rouges **plausibles** sur du code sain.

CE QUE LA RECIDIVE AJOUTE A LA REGLE, ET C'EST LA SEULE RAISON DE L'ECRIRE

La regle existait, **ecrite par moi**, apres l'avoir payee une premiere fois. Mais elle parlait du **controle negatif du CORRECTIF**.

Or **un document a ses gardes aussi, et les eprouver mute les memes fichiers**. Le geste est identique ; seule l'**intention** differe — et *l'intention n'est pas ce que le serveur du banc lit*.

=> **La regle se generalise : pendant une passe, aucun fichier servi ne bouge, quelle que soit la RAISON de le faire**. Controle negatif du code, gardes d'un PDF, essai rapide, mesure de curiosite — meme interdit.

=> **Une regle qu'on a ecrite soi-meme, apres l'avoir payee, ne suffit pas a empecher le geste**. Ce qui a limite les degats n'est pas la regle, c'est le reflexe : passe **arretee immediatement**, les trois fichiers **verifies restaures au diff**, puis **relancee de zero sans rien toucher**. Elle a rendu le total exactement predit. *Rien n'a ete conclu sur une passe faussee — c'est la seule chose qui compte une fois la faute commise*.

La section correspondante du catalogue de bugs a ete completee : elle portait trois cas, elle en porte un de plus, et c'est **celui de son propre auteur**.

7. Les mesures

preuve	resultat
Instantane (15 cles), avant / apres	identique octet pour octet , sha256 7a52c37da93e17a3
Controle negatif	10 mutations, TOUTES mordent — et le controle sain d'abord, a 0 rouge sur 15
Temoins	bloc CCXCIV , 15 temoins, dont 2 de perimetre
Source : le propriétaire est unique	4 occurrences (1 declaration + 3 appelants)
Source : les 3 divergences sont intactes	les trois formulations d'origine sont toujours la

LES DEUX MUTATIONS QUI COMPTENT LE PLUS

(1) **L'harmonisation interdite** — faire hériter la porte directe de `it.origine` : **1 rouge, exactement le témoin qui protège ce choix.**

(2) **Le débordement** — absorber `origine` ou `saisie` dans le propriétaire : **5 et 4 rouges**, dont les deux témoins de périmètre.

Une mutation qui ne mord nulle part désigne un témoin manquant, pas du code inutile. Ici, les deux frontières que Michel a posées sont tenues par des témoins qui rougissent.

8. Ce qui reste

sujet	etat
Les 6 sous-etapes restantes	auditées et réelles. /\ Le test d'entrée (compter les copies) leur a été appliqué — il ne l'avait pas été à l'écriture du plan
Les 4 harmonisations	décisions produites , elles attendent Michel. Critère : est-ce que ça modifie ce qui est ECRIT dans le journal alimentaire ?
Le hub et la douane	après 1b et 3, consigne explicite inchangée
Les favoris perdus entre onglets, l'écart 48,3 / 48	ouverts, non corrigés
L'historique, les migrations	non touchées

Document généré depuis le code servi — tous les décomptes cités sont recomptés à chaque génération (4 occurrences du propriétaire, 15 clés de sonde dont 3 pour les trois portes, 2 occurrences de la liste de colonnes CSV, 3 de son propriétaire commun). **Neuf gardes refusent de produire si un fait tombe** — dont un qui vérifie que `origine/saisie` ne sont **pas** entrés dans le propriétaire, et un qui vérifie que les **trois** formulations divergentes sont toujours là. Source : `tools/gen_1bii_pdf.py`.